



Faculty of Engineering and Technology
Electrical and Computer Engineering Department

ENCS4380

HW_2

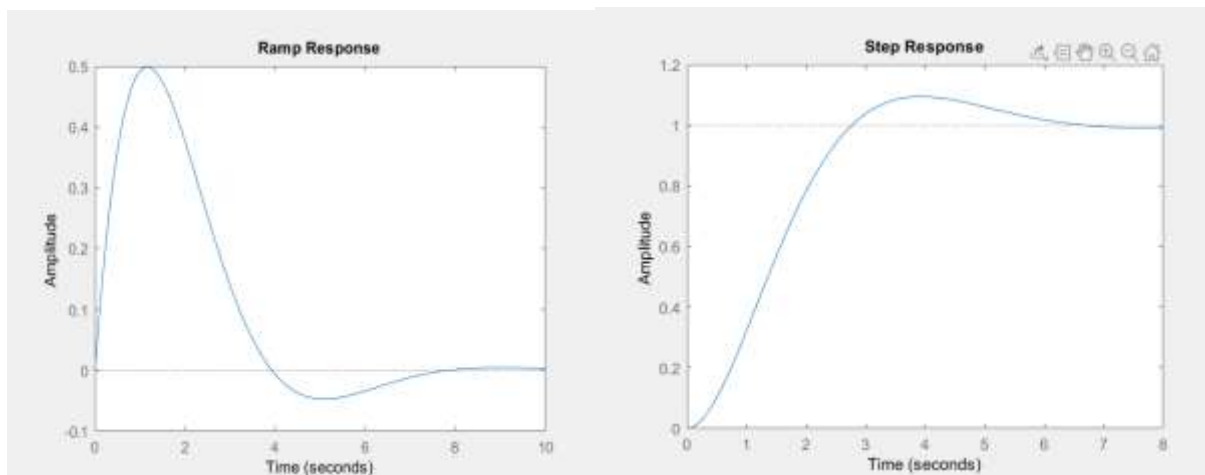
Prepared by

Aya Dahbour 1201738

Section number: 1

Question#1

```
Q1.m x +
1 % the parameters
2 - m = 1; % mass
3 - b = 1.2; % increase damping for less oscillation
4 - c = 1; % spring constant
5 % the transfer function
6 - num = [0 0 1];
7 - den = [m b c];
8 % step response
9 - figure;
10 - step(tf(num, den));
11 - title('Step Response');
12 % ramp response
13 - figure;
14 - impulse(tf(num, den));
15 - title('Ramp Response');
```



The nature of behavior in a system will be determined by the damping ratio B/M . Increased damping is ideal in that it reduces the overshoot in step responses, smoothens the responses to ramp inputs and also stabilizes the system - ultimately lowering the frequency response peak. Smaller damping may cause oscillations on both responses and gives a smaller peak which means that system stability may be compromised. The damping coefficient is determined by balancing the reaction speed with the system stability.

Question#2

- Concept of Operation

Smartphone accelerometers use MEMS technology with colorful seeing mechanisms similar as capacitive, piezoresistive, and piezoelectric to descry direct acceleration and stir. Capacitive accelerometers, generally set up in smartphones, calculate on changes in capacitance due to movement, while piezoelectric detectors descry subtle changes in pressure.

- Technology Used in Manufacturing

Manufacturing involves the use of capacitive, piezoelectric, and piezoresistive technologies for accelerometers. Gyroscopes, on the other hand, employ MEMS technology, microscale wobbling structures, and the Coriolis effect.

- Stationary and Dynamic Characteristics

stationary characteristics (perceptivity, range, resolution, noise, and frequency response) are pivotal for both accelerometers and gyroscopes. Capacitive accelerometers are popular in smartphones, piezoelectric detectors capture subtle changes, and piezoresistive detectors balance perceptivity and responsiveness. Gyroscopes partake stationary characteristics with accelerometers.

- Simple Interface for measures

Accelerometers and gyroscopes give precious data through stoner-friendly APIs or software libraries. inventors can fluently interact with these detectors using standard programming languages and protocols. The flawless integration of detector data into operations, similar as pedometers, gesture recognition, and stoked reality, is eased by accessible and well- proved interfaces. This stoner-friendly approach enhances the overall experience for inventors using detector capabilities on smartphones.

Question#3

```
Q3.m x +
1 % parameters for the sinusoidal temperature variation
2 - A = (350 - 150) / 2;
3 - offset = 150 + A;
4 - freq = 1/10;
5
6 % time vector from 0 to 30 sec
7 - t = 0:0.1:30; % with a step of 0.1 sec
8
9 % generate the temperature signal
10 - temperature = A * sin(2 * pi * freq * t) + offset;
11
12 % calculate and display the maximum and minimum temperatures
13 - max_temp = max(temperature);
14 - min_temp = min(temperature);
15
16 - disp(['Max Temperature: ' num2str(max_temp)]);
17 - disp(['Min Temperature: ' num2str(min_temp)]);
18
```

➤ After run this code in MATLAB I got these results:

```
>> Q3
Max Temperature: 350
Min Temperature: 150
fx >>
```

Question#4

```
1 - signalVoltageLevel = 6e-6; % signal voltage level in volts
2 - noiseVoltageLevel_input = 1e-6; % noise voltage level in volts
3 - amplifierGain = 80;
4 - additionalOut_noise = 2e-6; % noise added at the output
5
6 % calculate output noise after amplification
7 - noiseVoltageLevel_output = amplifierGain * noiseVoltageLevel_input;
8
9 % calculate signal-to-noise ratio at the output
10 - signal_to_noise_ratio_output = signalVoltageLevel / ...
11     sqrt(noiseVoltageLevel_output^2 + additionalOut_noise^2);
12
13 - disp(['Signal-to-Noise Ratio at Output: ' num2str(signal_to_noise_ratio_output)]);
14
15 ~~~~~
Signal-to-Noise Ratio at Output: 0.074977
fx >>
```

Question#5

```
1 % given data
2 Vs = 3.3;
3 sensitivityCoefficient = 0.004;
4 offsetCoefficient = -0.04;
5
6 % (1) calculate the sensor's nominal sensitivity and offset
7 nominalSensitivity = sensitivityCoefficient * Vs;
8 nominalOffset = offsetCoefficient * Vs;
9
10 disp(['Nominal Sensitivity: ' num2str(nominalSensitivity) ' V/kPa']);
11 disp(['Nominal Offset: ' num2str(nominalOffset) ' V']);
12
13 % (2) find the maximum and minimum absolute output error
14 Vs_min = 2.85;
15 Vs_max = 3.5;
16 inputPressure = 100; % in kPa
17
18 max_output_error = sensitivityCoefficient * (Vs_max - inputPressure) - nominalOffset;
19 min_output_error = sensitivityCoefficient * (Vs_min - inputPressure) - nominalOffset;
20
21 disp(['Max Output Error: ' num2str(max_output_error) ' V']);
22 disp(['Min Output Error: ' num2str(min_output_error) ' V']);
```

➤ The code output:

```
Nominal Sensitivity: 0.0132 V/kPa
Nominal Offset: -0.132 V
Max Output Error: -0.254 V
Min Output Error: -0.2566 V
% ~~
```

Question#6

```
1 % given values
2 Vin = 5;
3 R1 = 10e3;
4 R2Bright = 700; % R2 in bright light condition
5 R2Dark = 100e3; % R2 in dark condition
6
7 % calculate output voltage in bright light condition
8 VoutBright = Vin * R2Bright / (R1 + R2Bright);
9
10 % calculate output voltage in dark condition
11 VoutDark = Vin * R2Dark / (R1 + R2Dark);
12
13 % display results
14 disp(['Output Voltage in Bright Light: ' num2str(VoutBright) ' V']);
15 disp(['Output Voltage in Dark: ' num2str(VoutDark) ' V']);
16
17 % ~~
18
19 Output Voltage in Bright Light: 0.3271 V
20 Output Voltage in Dark: 4.5455 V
```

Question#7

```
1 - timeConstant = 38;           % time constant of the thermometer in seconds
2 - periodicChange = 3 / 60;    % periodic temperature change rate in changes per second
3
4 - % Calculate the delay introduced by the thermometer
5 - delay = timeConstant / (1 - exp(-periodicChange / timeConstant));
6
7 - disp(['Delay introduced by the thermometer: ' num2str(delay) ' seconds']);
```

➤ The code output:

```
Delay introduced by the thermometer: 28899.0042 seconds
>>
```

Question#8

```
1 - strain = 160e-6;           % in  $\mu\text{m} \cdot \text{m}^{-1}$ 
2 - nominalResistance = 120;
3 - GF_metallic = 2.13;        % GF for metallic gauge
4 - GF_semiconductor = -161;   % GF for semiconductor gauge
5
6 - % calculate resistance changes for both gauges
7 - delta_R_metallic = GF_metallic * strain * nominalResistance;
8 - delta_R_semiconductor = GF_semiconductor * strain * nominalResistance;
9
0 - disp(['Resistance Change for Metallic Gauge: ' num2str(delta_R_metallic) ' ohms']);
1 - disp(['Resistance Change for Semiconductor Gauge: ' num2str(delta_R_semiconductor) ' ohms']);
```

➤ These are the values that resulted:

```
// x~
Resistance Change for Metallic Gauge: 0.040896 ohms
Resistance Change for Semiconductor Gauge: -3.0912 ohms
% ~\n
```

Question#9

```
1 - % given parameters
2 - lf = 300e-6; % finger length (m)
3 - lm = 280e-6; % mass length
4 - n = 100;     % # of fingers
5 - df = 1e-6;   % air gap
6 - epsilonR = 7; % relative permittivity
7 - deltaC = 50.34e-12; % measured capacitance change in farads
8
9 - % calculate the angle theta
0 - theta = atand((deltaC / (2 * n * epsilonR * df)) / (pi * (lf + lm)));
1
2 - disp(['Angle Theta: ' num2str(theta) ' degrees']);
```

```
..
Angle Theta: 0.0011307 degrees
;>>
```

Question#10

```
1 - desiredFrequency = 200;    % maximum input signal frequency
2 - dampingRatio = 0.6;        % damping ratio
3
4   % calculate the natural frequency (wn)
5 - wn = desiredFrequency / sqrt(1 - dampingRatio^2);
6
7   % print results
8 - disp(['Natural Frequency (wn): ' num2str(wn) ' rad/s']);
9 - disp(['Maximum Input Signal Frequency: ' num2str(desiredFrequency) ' Hz']);
10
```

```
Natural Frequency (wn): 250 rad/s
Maximum Input Signal Frequency: 200 Hz
^^
```

Question#11

I didn't know exactly which program you wanted the solution on, but I used tinkercad.

